

Spock testing revisited for Grails 7/8

A walkthrough of Grails testing support,
including the new modules and features in Grails 7 and 8

Mattias Reichel · Apache Grails PMC
Community over Code Europe · Glasgow · October 2026

whoami

- **Mattias Reichel**, Sweden
- Apache Grails committer & PMC member
- Grailsy things I've touched lately
  - Oct 2024 Containerized Geb
  - 2025–26 Grails Spring Security Plugins
  - Feb 2026 Grails Website plugin page
  - Mar 2026 HttpClient Testing Support
  - Sep 2026 Grails Website search
  - Now First conference talk



github.com/matrei
x.com/mattias_reichel
grails.apache.org

Three generations, one test

1 · Base classes | 2 · Mixins | 3 · Traits

2009 · Grails 1.1

JUnit 3 via `GroovyTestCase` · `MockUtils` · Spock is a plugin

```
class BookControllerTests extends ControllerUnitTestCase {

    void testIndexAction() {
        mockDomain(Book, [
            new Book(title: 'Geb')
        ])

        def model = controller.index()

        assert model.books.size() == 1
    }
}
```

Three generations, one test

1 · Base classes | **2 · Mixins** | 3 · Traits

2011 · Grails 2.0 – 3.2

AST transforms weave the mixins in · JUnit 4 or Spock

```
@Mock(Book)
@TestFor(BookController)
class BookControllerSpec extends Specification {

    void 'the index action lists books'() {
        given: 'a saved book'
            new Book(title: 'Geb').save()

        when: 'the index action is called'
            def model = controller.index()

        then: 'the book is in the model'
            model.books.size() == 1
    }
}
```

Three generations, one test

1 · Base classes | 2 · Mixins | **3 · Traits**

2017 · Grails 3.3 – 8

Plain traits · Spock extensions, no AST · what you write today

```
class BookControllerSpec extends Specification
        implements DataTest, ControllerUnitTest<BookController> {

    void setupSpec() { mockDomain(Book) }

    void 'the index action lists books'() {
        given: 'a saved book'
            new Book(title: 'Geb').save()

        when: 'the index action is called'
            def model = controller.index()

        then: 'the book is in the model'
            model.books.size() == 1
    }
}
```

History of the current generation

2017 · Grails 3.3

Spock	1.1
Geb	1.1
Groovy	2.4
Gradle	3.5

- Trait-based testing support replaces the mixin framework
- ControllerUnitTest, DomainUnitTest, DataTest & friends

2019 · Grails 4

Spock	1.2
Geb	2.3
Groovy	2.5
Gradle	5.1

2021 · Grails 5

Spock	2.0
Geb	2.3
Groovy	3.0
Gradle	7.2

2023 · Grails 6

Spock	2.1
Geb	5.1
Groovy	3.0
Gradle	7.6

2025 · Grails 7

Spock	2.3
Geb	8.0
Groovy	4.0
Gradle	8.14

- Geb in containers
- @DatabaseCleanup
- HttpClientSupport
- MongoDB containers

2026 · Grails 8

Spock	2.4
Geb	8.0
Groovy	5.1
Gradle	9.8

- New modules: latency, embedded MongoDB
- Redis containers (8.0.1)

Versions read from a generated app's resolved test classpath and Gradle wrapper, GA release of each line: 3.3.0, 4.0.0, 5.0.0, 6.0.0, 7.0.0 and 8.0.0.

History of the current generation

2017 · Grails 3.3

Spock	1.1
Geb	1.1
Groovy	2.4
Gradle	3.5

- Trait-based testing support replaces the mixin framework
- ControllerUnitTest, DomainUnitTest, DataTest & friends

2019 · Grails 4

Spock	1.2
Geb	2.3
Groovy	2.5
Gradle	5.1

2021 · Grails 5

Spock	2.0
Geb	2.3
Groovy	3.0
Gradle	7.2

2023 · Grails 6

Spock	2.1
Geb	5.1
Groovy	3.0
Gradle	7.6

2025 · Grails 7

Spock	2.3
Geb	8.0
Groovy	4.0
Gradle	8.14

- Geb in containers
- @DatabaseCleanup
- HttpClientSupport
- MongoDB containers

2026 · Grails 8

Spock	2.4
Geb	8.0
Groovy	5.1
Gradle	9.8

- New modules: latency, embedded MongoDB
- Redis containers (8.0.1)

The traits didn’t change. The runtime under them did.

And a few new pieces arrived that are easy to miss.

Versions read from a generated app’s resolved test classpath and Gradle wrapper, GA release of each line: 3.3.0, 4.0.0, 5.0.0, 6.0.0, 7.0.0 and 8.0.0.

The testing landscape

Where tests live and what they pull in

Two phases out of the box

Unit

`src/test/groovy`
`./gradlew test`

traits · in-memory datastore · a tiny application context · no server

Integration

`src/integration-test/groovy`
`./gradlew integrationTest`

@Integration · the real app on a random port · real database · **browser tests with Geb live here too**

Your own phases

OPT-IN

```
src/functional-test/groovy
testPhases {
    functionalTest { }
}
```

declared with the Grails Gradle plugin's `testPhases` (7.1), e.g. to keep Docker-bound browser specs apart

Two phases out of the box

Unit

`src/test/groovy`
`./gradlew test`

traits · in-memory datastore · a tiny application context · no server

---

milliseconds per test
thousands of them

Integration

`src/integration-test/groovy`
`./gradlew integrationTest`

@Integration · the real app on a random port · real database · **browser tests with Geb live here too**

Your own phases

OPT-IN

```
src/functional-test/groovy
testPhases {
    functionalTest { }
}
```

declared with the Grails Gradle plugin's `testPhases` (7.1), e.g. to keep Docker-bound browser specs apart

Two phases out of the box

Unit

`src/test/groovy`
`./gradlew test`

traits · in-memory datastore · a tiny application context · no server

milliseconds per test
thousands of them

Integration

`src/integration-test/groovy`
`./gradlew integrationTest`

@Integration · the real app on a random port · real database · **browser tests with Geb live here too**

seconds per class
the app boots once per JVM

Your own phases

OPT-IN

```
src/functional-test/groovy
testPhases {
    functionalTest { }
}
```

declared with the Grails Gradle plugin's `testPhases` (7.1), e.g. to keep Docker-bound browser specs apart

Two phases out of the box

Unit

`src/test/groovy`
`./gradlew test`

traits · in-memory datastore · a tiny application context · no server

milliseconds per test
thousands of them

Integration

`src/integration-test/groovy`
`./gradlew integrationTest`

@Integration · the real app on a random port · real database · **browser tests with Geb live here too**

seconds per class
the app boots once per JVM

Your own phases

OPT-IN

```
src/functional-test/groovy
testPhases {
    functionalTest { }
}
```

declared with the Grails Gradle plugin's `testPhases` (7.1), e.g. to keep Docker-bound browser specs apart

same @Integration underneath
just a different task and classpath

Unit testing

`implements XxxUnitTest<T>` and you're done

`GrailsUnitTest`: the base of everything

```
class MySpec extends Specification implements GrailsUnitTest {

    // a tiny application context, built once per spec by a Spock global extension
    // applicationContext, grailsApplication, config, messageSource

    def beans = {
        // compiled into a nested @Configuration class, e.g.
        // bean('mathHelper', RushHelper)
    }

    Closure doWithConfig() {{ config →
        // override configuration for this spec only
    }}
}
```

Every other unit trait extends this one. You rarely implement it directly, but its hooks are what you reach for when a test needs a bean or a config value.

`GrailsUnitTest`: the base of everything

```
class MySpec extends Specification implements GrailsUnitTest {

    // a tiny application context, built once per spec by a Spock global extension
    // applicationContext, grailsApplication, config, messageSource

    def beans = {
        // compiled into a nested @Configuration class, e.g.
        // bean('mathHelper', RushHelper)
    }

    Closure doWithConfig() {{ config →
        // override configuration for this spec only
    }}
}
```

Every other unit trait extends this one. You rarely implement it directly, but its hooks are what you reach for when a test needs a bean or a config value.

`GrailsUnitTest`: the base of everything

```
class MySpec extends Specification implements GrailsUnitTest {

    // a tiny application context, built once per spec by a Spock global extension
    // applicationContext, grailsApplication, config, messageSource

    def beans = {
        // compiled into a nested @Configuration class, e.g.
        // bean('mathHelper', RushHelper)
    }

    Closure doWithConfig() {{ config →
        // override configuration for this spec only
    }}
}
```

Every other unit trait extends this one. You rarely implement it directly, but its hooks are what you reach for when a test needs a bean or a config value.

`GrailsUnitTest`: the base of everything

```
class MySpec extends Specification implements GrailsUnitTest {

    // a tiny application context, built once per spec by a Spock global extension
    // applicationContext, grailsApplication, config, messageSource

    def beans = {
        // compiled into a nested @Configuration class, e.g.
        // bean('mathHelper', RushHelper)
    }

    Closure doWithConfig() {{ config →
        // override configuration for this spec only
    }}
}
```

Every other unit trait extends this one. You rarely implement it directly, but its hooks are what you reach for when a test needs a bean or a config value.

`GrailsUnitTest`: the base of everything

```
class MySpec extends Specification implements GrailsUnitTest {

    // a tiny application context, built once per spec by a Spock global extension
    // applicationContext, grailsApplication, config, messageSource

    def beans = {
        // compiled into a nested @Configuration class, e.g.
        // bean('mathHelper', RushHelper)
    }

    Closure doWithConfig() {{ config →
        // override configuration for this spec only
    }}
}
```

Every other unit trait extends this one. You rarely implement it directly, but its hooks are what you reach for when a test needs a bean or a config value.

One trait per artefact

Artefact	Trait	You get
Service	`ServiceUnitTest<T>`	`service`, a real Spring bean
Domain class	`DomainUnitTest<T>`	`domain`, validation against the real constraints
Controller	`ControllerUnitTest<T>`	`controller`, `view`, `model`, plus request, response, params, flash, session
Tag library	`TagLibUnitTest<T>`	`tagLib`, `applyTemplate()`
Interceptor	`InterceptorUnitTest<T>`	`interceptor`, `withRequest()`
URL mappings	`UrlMappingsUnitTest<T>`	`assertForwardUrlMapping()`, `assertReverseUrlMapping()`
Add for GORM	`DataTest`	`mockDomain()`, an in-memory datastore

ControllerUnitTest<T>

controller · view · model

WEB · request · response · params · flash · session · status · views · servletContext

```
class PersonControllerSpec extends Specification
        implements ControllerUnitTest<PersonController>, DataTest {

    void setupSpec() { mockDomain(Person) }

    void 'index lists people sorted by last name'() {
        given: 'two people'
            new Person(firstName: 'Ada', lastName: 'Lovelace').save()
            new Person(firstName: 'Grace', lastName: 'Hopper').save()

        expect: 'index sorts them by last name'
            controller.index().people*.lastName == ['Hopper', 'Lovelace']
    }

    void 'show responds with JSON'() {
        given: 'a person and a JSON request'
            def ada = new Person(firstName: 'Ada', lastName: 'Lovelace').save()
            response.format = 'json'

        when: 'the show action is called'
            controller.show(ada.id)

        then: 'the person is rendered as JSON'
            status == 200
            response.json.firstName == 'Ada'
    }
}
```

ControllerUnitTest<T>

controller · view · model

WEB · request · response · params · flash · session · status · views · servletContext

```
class PersonControllerSpec extends Specification
        implements ControllerUnitTest<PersonController>, DataTest {

    void setupSpec() { mockDomain(Person) }

    void 'index lists people sorted by last name'() {
        given: 'two people'
            new Person(firstName: 'Ada', lastName: 'Lovelace').save()
            new Person(firstName: 'Grace', lastName: 'Hopper').save()

        expect: 'index sorts them by last name'
            controller.index().people*.lastName == ['Hopper', 'Lovelace']
    }

    void 'show responds with JSON'() {
        given: 'a person and a JSON request'
            def ada = new Person(firstName: 'Ada', lastName: 'Lovelace').save()
            response.format = 'json'

        when: 'the show action is called'
            controller.show(ada.id)

        then: 'the person is rendered as JSON'
            status == 200
            response.json.firstName == 'Ada'
    }
}
```

ControllerUnitTest<T>

controller · view · model

WEB · request · response · params · flash · session · status · views · servletContext

```
class PersonControllerSpec extends Specification
        implements ControllerUnitTest<PersonController>, DataTest {

    void setupSpec() { mockDomain(Person) }

    void 'index lists people sorted by last name'() {
        given: 'two people'
            new Person(firstName: 'Ada', lastName: 'Lovelace').save()
            new Person(firstName: 'Grace', lastName: 'Hopper').save()

        expect: 'index sorts them by last name'
            controller.index().people*.lastName == ['Hopper', 'Lovelace']
    }

    void 'show responds with JSON'() {
        given: 'a person and a JSON request'
            def ada = new Person(firstName: 'Ada', lastName: 'Lovelace').save()
            response.format = 'json'

        when: 'the show action is called'
            controller.show(ada.id)

        then: 'the person is rendered as JSON'
            status == 200
            response.json.firstName == 'Ada'
    }
}
```

ControllerUnitTest<T>

controller · view · model

WEB · request · response · params · flash · session · status · views · servletContext

```
class PersonControllerSpec extends Specification
        implements ControllerUnitTest<PersonController>, DataTest {

    void setupSpec() { mockDomain(Person) }

    void 'index lists people sorted by last name'() {
        given: 'two people'
            new Person(firstName: 'Ada', lastName: 'Lovelace').save()
            new Person(firstName: 'Grace', lastName: 'Hopper').save()

        expect: 'index sorts them by last name'
            controller.index().people*.lastName == ['Hopper', 'Lovelace']
    }

    void 'show responds with JSON'() {
        given: 'a person and a JSON request'
            def ada = new Person(firstName: 'Ada', lastName: 'Lovelace').save()
            response.format = 'json'

        when: 'the show action is called'
            controller.show(ada.id)

        then: 'the person is rendered as JSON'
            status == 200
            response.json.firstName == 'Ada'
    }
}
```

ControllerUnitTest<T>

controller view model

WEB request response params flash session status views servletContext

```
class PersonControllerSpec extends Specification
        implements ControllerUnitTest<PersonController>, DataTest {

    void setupSpec() { mockDomain(Person) }

    void 'index lists people sorted by last name'() {
        given: 'two people'
            new Person(firstName: 'Ada', lastName: 'Lovelace').save()
            new Person(firstName: 'Grace', lastName: 'Hopper').save()

        expect: 'index sorts them by last name'
            controller.index().people*.lastName == ['Hopper', 'Lovelace']
    }

    void 'show responds with JSON'() {
        given: 'a person and a JSON request'
            def ada = new Person(firstName: 'Ada', lastName: 'Lovelace').save()
            response.format = 'json'

        when: 'the show action is called'
            controller.show(ada.id)

        then: 'the person is rendered as JSON'
            status == 200
            response.json.firstName == 'Ada'
    }
}
```

ControllerUnitTest<T>

controller · view · model

WEB · request · response · params · flash · session · status · views · servletContext

```
class PersonControllerSpec extends Specification
        implements ControllerUnitTest<PersonController>, DataTest {

    void setupSpec() { mockDomain(Person) }

    void 'index lists people sorted by last name'() {
        given: 'two people'
            new Person(firstName: 'Ada', lastName: 'Lovelace').save()
            new Person(firstName: 'Grace', lastName: 'Hopper').save()

        expect: 'index sorts them by last name'
            controller.index().people*.lastName == ['Hopper', 'Lovelace']
    }

    void 'show responds with JSON'() {
        given: 'a person and a JSON request'
            def ada = new Person(firstName: 'Ada', lastName: 'Lovelace').save()
            response.format = 'json'

        when: 'the show action is called'
            controller.show(ada.id)

        then: 'the person is rendered as JSON'
            status == 200
            response.json.firstName == 'Ada'
    }
}
```

ControllerUnitTest<T>

controller · view · model

WEB · request · response · params · flash · session · status · views · servletContext

```
class PersonControllerSpec extends Specification
        implements ControllerUnitTest<PersonController>, DataTest {

    void setupSpec() { mockDomain(Person) }

    void 'index lists people sorted by last name'() {
        given: 'two people'
            new Person(firstName: 'Ada', lastName: 'Lovelace').save()
            new Person(firstName: 'Grace', lastName: 'Hopper').save()

        expect: 'index sorts them by last name'
            controller.index().people*.lastName == ['Hopper', 'Lovelace']
    }

    void 'show responds with JSON'() {
        given: 'a person and a JSON request'
            def ada = new Person(firstName: 'Ada', lastName: 'Lovelace').save()
            response.format = 'json'

        when: 'the show action is called'
            controller.show(ada.id)

        then: 'the person is rendered as JSON'
            status == 200
            response.json.firstName == 'Ada'
    }
}
```

ControllerUnitTest<T>

controller · view · model

WEB · request · response · params · flash · session · status · views · servletContext

```
class PersonControllerSpec extends Specification
        implements ControllerUnitTest<PersonController>, DataTest {

    void setupSpec() { mockDomain(Person) }

    void 'index lists people sorted by last name'() {
        given: 'two people'
            new Person(firstName: 'Ada', lastName: 'Lovelace').save()
            new Person(firstName: 'Grace', lastName: 'Hopper').save()

        expect: 'index sorts them by last name'
            controller.index().people*.lastName == ['Hopper', 'Lovelace']
    }

    void 'show responds with JSON'() {
        given: 'a person and a JSON request'
            def ada = new Person(firstName: 'Ada', lastName: 'Lovelace').save()
            response.format = 'json'

        when: 'the show action is called'
            controller.show(ada.id)

        then: 'the person is rendered as JSON'
            status == 200
            response.json.firstName == 'Ada'
    }
}
```

Also in the box

`JsonViewUnitTest`

```
class BookViewSpec extends Specification
        implements JsonViewUnitTest {

    void 'renders a book'() {
        when:
            def result = render(view: '/book/_book',
                    model: [book: new Book(title: 'Geb')])

        then:
            result.json.title == 'Geb'
    }
}
```

module `grails-testing-support-views-gson`

`HibernateSpec`

```
class BookHibernateSpec extends HibernateSpec {

    List<Class> getDomainClasses() { [Book] }

    void 'real Hibernate, no app booted'() {
        expect:
            Book.count() == 0
    }
}
```

module `grails-data-hibernate5` ; H2 in memory, real mapping, real SQL

Also: `AutowiredTest` for plain beans, `@Rollback` and `@Transactional` in HibernateSpec specs.

Integration testing

The real app, the real database, one JVM

`@Integration`: what you write, what compiles

```
@Integration
class PersonServiceIntegrationSpec extends Specification {

    PersonService personService

    void 'the real service is wired'() {
        expect:
            personService.welcome(1L) == 'Hello, Ada Lovelace!'
    }
}
```

`@Integration`: what you write, what compiles

```
@SpringBootTest(webEnvironment = RANDOM_PORT, useMainMethod = ALWAYS)
@ContextConfiguration(loader = GrailsApplicationContextLoader, classes = Application)
class PersonServiceIntegrationSpec extends Specification {

    PersonService personService

    void 'the real service is wired'() {
        expect:
            personService.welcome(1L) == 'Hello, Ada Lovelace!'
    }
}
```

`@Integration`: what you write, what compiles

```
@SpringBootTest(webEnvironment = RANDOM_PORT, useMainMethod = ALWAYS)
@ContextConfiguration(loader = GrailsApplicationContextLoader, classes = Application)
class PersonServiceIntegrationSpec extends Specification implements ApplicationContextAware {

    @Value('${local.server.port}')
    Integer serverPort

    PersonService personService   // autowired by name from the real context

    void 'the real service is wired'() {
        expect:
            personService.welcome(1L) == 'Hello, Ada Lovelace!'
    }
}
```

`@Rollback`: the classic

```groovy
@Rollback
@Integration
class PersonServiceIntegrationSpec extends Specification {

    PersonService personService

    void 'the real service and the real GreetingService are wired'() {
        given: 'a person created through the real service'
            def ada = personService.create('Ada', 'Lovelace')

        expect: 'the real GreetingService greets by full name'
            personService.welcome(ada.id) == 'Hello, Ada Lovelace!'
            Person.count() == 1
    }

    void 'the previous test was rolled back'() {
        expect: 'nothing survived the rollback'
            Person.count() == 0
    }
}
```

`@Rollback`: the classic

```groovy
@Rollback
@Integration
class PersonServiceIntegrationSpec extends Specification {

    PersonService personService

    void 'the real service and the real GreetingService are wired'() {
        given: 'a person created through the real service'
            def ada = personService.create('Ada', 'Lovelace')

        expect: 'the real GreetingService greets by full name'
            personService.welcome(ada.id) == 'Hello, Ada Lovelace!'
            Person.count() == 1
    }

    void 'the previous test was rolled back'() {
        expect: 'nothing survived the rollback'
            Person.count() == 0
    }
}
```

`@Rollback`: the classic

```
@Rollback
@Integration
class PersonServiceIntegrationSpec extends Specification {

    PersonService personService

    void 'the real service and the real GreetingService are wired'() {
        given: 'a person created through the real service'
            def ada = personService.create('Ada', 'Lovelace')

        expect: 'the real GreetingService greets by full name'
            personService.welcome(ada.id) == 'Hello, Ada Lovelace!'
            Person.count() == 1
    }

    void 'the previous test was rolled back'() {
        expect: 'nothing survived the rollback'
            Person.count() == 0
    }
}
```

`@Rollback`: the classic

```
@Rollback
@Integration
class PersonServiceIntegrationSpec extends Specification {

    PersonService personService

    void 'the real service and the real GreetingService are wired'() {
        given: 'a person created through the real service'
            def ada = personService.create('Ada', 'Lovelace')

        expect: 'the real GreetingService greets by full name'
            personService.welcome(ada.id) == 'Hello, Ada Lovelace!'
            Person.count() == 1
    }

    void 'the previous test was rolled back'() {
        expect: 'nothing survived the rollback'
            Person.count() == 0
    }
}
```

`@Rollback`: the classic

```
@Rollback
@Integration
class PersonServiceIntegrationSpec extends Specification {

    PersonService personService

    void 'the real service and the real GreetingService are wired'() {
        given: 'a person created through the real service'
            def ada = personService.create('Ada', 'Lovelace')

        expect: 'the real GreetingService greets by full name'
            personService.welcome(ada.id) == 'Hello, Ada Lovelace!'
            Person.count() == 1
    }

    void 'the previous test was rolled back'() {
        expect: 'nothing survived the rollback'
            Person.count() == 0
    }
}
```

`@Rollback`: the classic

```
@Rollback
@Integration
class PersonServiceIntegrationSpec extends Specification {

    PersonService personService

    void 'the real service and the real GreetingService are wired'() {
        given: 'a person created through the real service'
            def ada = personService.create('Ada', 'Lovelace')

        expect: 'the real GreetingService greets by full name'
            personService.welcome(ada.id) == 'Hello, Ada Lovelace!'
            Person.count() == 1
    }

    void 'the previous test was rolled back'() {
        expect: 'nothing survived the rollback'
            Person.count() == 0
    }
}
```

@DatabaseCleanup

NEW IN 7.0.8

```
@Stepwise
@Integration
@DatabaseCleanup
class PersonCleanupSpec extends Specification implements HttpClientSupport {

    void 'the running app sees committed data'() {
        given: 'a person, committed in its own transaction'
            Person.withNewTransaction {
                new Person(firstName: 'Ada', lastName: 'Lovelace').save(failOnError: true)
            }

        expect: 'the app lists the person over HTTP'
            http('/people').assertContains(200, 'Lovelace')
    }

    void 'the tables were truncated after the previous test'() {
        expect: 'the app lists nobody'
            http('/people').assertNotContains(200, 'Lovelace')
    }
}
```

@DatabaseCleanup

NEW IN 7.0.8

```
@Stepwise
@Integration
@DatabaseCleanup
class PersonCleanupSpec extends Specification implements HttpClientSupport {

    void 'the running app sees committed data'() {
        given: 'a person, committed in its own transaction'
            Person.withNewTransaction {
                new Person(firstName: 'Ada', lastName: 'Lovelace').save(failOnError: true)
            }

        expect: 'the app lists the person over HTTP'
            http('/people').assertContains(200, 'Lovelace')
    }

    void 'the tables were truncated after the previous test'() {
        expect: 'the app lists nobody'
            http('/people').assertNotContains(200, 'Lovelace')
    }
}
```

@DatabaseCleanup

NEW IN 7.0.8

```
@Stepwise
@Integration
@DatabaseCleanup
class PersonCleanupSpec extends Specification implements HttpClientSupport {

    void 'the running app sees committed data'() {
        given: 'a person, committed in its own transaction'
            Person.withNewTransaction {
                new Person(firstName: 'Ada', lastName: 'Lovelace').save(failOnError: true)
            }

        expect: 'the app lists the person over HTTP'
            http('/people').assertContains(200, 'Lovelace')
    }

    void 'the tables were truncated after the previous test'() {
        expect: 'the app lists nobody'
            http('/people').assertNotContains(200, 'Lovelace')
    }
}
```

@DatabaseCleanup

NEW IN 7.0.8

```
@Stepwise
@Integration
@DatabaseCleanup
class PersonCleanupSpec extends Specification implements HttpClientSupport {

    void 'the running app sees committed data'() {
        given: 'a person, committed in its own transaction'
            Person.withNewTransaction {
                new Person(firstName: 'Ada', lastName: 'Lovelace').save(failOnError: true)
            }

        expect: 'the app lists the person over HTTP'
            http('/people').assertContains(200, 'Lovelace')
    }

    void 'the tables were truncated after the previous test'() {
        expect: 'the app lists nobody'
            http('/people').assertNotContains(200, 'Lovelace')
    }
}
```

`@DatabaseCleanup`: the details

```
// only after this feature method
@DatabaseCleanup
void 'imports the catalogue'() { … }

// name data sources, and their type
@DatabaseCleanup(['dataSource:h2',
                  'other:postgresql'])

// once, after the whole spec
@DatabaseCleanup(cleanupAfterSpec = true)
class ImportSpec extends Specification { … }
```

`@DatabaseCleanup`: the details

```
// only after this feature method
@DatabaseCleanup
void 'imports the catalogue'() { … }

// name data sources, and their type
@DatabaseCleanup(['dataSource:h2',
                  'other:postgresql'])

// once, after the whole spec
@DatabaseCleanup(cleanupAfterSpec = true)
class ImportSpec extends Specification { … }
```

- **Requires** `@Integration`, it needs the application context

`@DatabaseCleanup`: the details

```
// only after this feature method
@DatabaseCleanup
void 'imports the catalogue'() { … }

// name data sources, and their type
@DatabaseCleanup(['dataSource:h2',
                  'other:postgresql'])

// once, after the whole spec
@DatabaseCleanup(cleanupAfterSpec = true)
class ImportSpec extends Specification { … }
```

- **Requires** `@Integration`, it needs the application context
- **Truncates** `BootStrap` **data too**: create what your test needs

`@DatabaseCleanup`: the details

```
// only after this feature method
@DatabaseCleanup
void 'imports the catalogue'() { … }

// name data sources, and their type
@DatabaseCleanup(['dataSource:h2',
                  'other:postgresql'])

// once, after the whole spec
@DatabaseCleanup(cleanupAfterSpec = true)
class ImportSpec extends Specification { … }
```

- **Requires** `@Integration`, it needs the application context
- **Truncates** `BootStrap` **data too**: create what your test needs
- **One module per database**: PostgreSQL uses `TRUNCATE … RESTART IDENTITY CASCADE`; H2 switches referential integrity off and keeps its sequences; a `DatabaseCleaner` SPI for others

`@DatabaseCleanup`: the details

```
// only after this feature method
@DatabaseCleanup
void 'imports the catalogue'() { … }

// name data sources, and their type
@DatabaseCleanup(['dataSource:h2',
                  'other:postgresql'])

// once, after the whole spec
@DatabaseCleanup(cleanupAfterSpec = true)
class ImportSpec extends Specification { … }
```

- **Requires** `@Integration`, it needs the application context
- **Truncates** `BootStrap` **data too**: create what your test needs
- **One module per database**: PostgreSQL uses `TRUNCATE … RESTART IDENTITY CASCADE`; H2 switches referential integrity off and keeps its sequences; a `DatabaseCleaner` SPI for others
- Set the `grails.testing.cleanup.debug` system property to log each cleanup: every table and its row count

HttpClientSupport

NEW IN 7.1

1 · GET | 2 · POST | 3 · XML

```
@Integration
class BookApiSpec extends Specification implements HttpClientSupport {

    void 'the app reports that it is up'() {
        when: 'the health endpoint is called'
            def response = http('/actuator/health', Accept: 'application/json')

        then: 'status, content type and body are checked one by one'
            response.assertStatus(200)
                    .assertHeaders('Content-Type': 'application/json')
                    .assertContains('UP')

        and: 'it can all be checked in one call'
            response.assertContains(200, 'Content-Type': 'application/json', 'UP')
    }
```

HttpClientSupport

NEW IN 7.1

1 · GET | **2 · POST** | 3 · XML

```
void 'POST creates a book'() {
    when: 'a book is posted as JSON'
        def response = httpPostJson(
                '/api/books',
                [title: 'Groovy in Action', author: 'Paul King']
        )

    then: 'it is created'
        response.assertJsonContains(
                201,
                [title: 'Groovy in Action']
        )
}
```

HttpClientSupport

NEW IN 7.1

1 · GET 2 · POST **3 · XML**

```
void 'POST and verify XML'() {
    when: 'a book is posted as XML, built with a MarkupBuilder closure'
        def response = httpPostXml('/api/books', Accept: 'application/xml') {
            book {
                title('Grails in Action')
                author('Glen Smith')
            }
        }

    then: 'the XML response is checked through GPath'
        response.assertStatus(201)
        with(response.xml()) {
            title.text() == 'Grails in Action'
            author.text() == 'Glen Smith'
        }
}
```

`HttpClientSupport`: more of the API

```
// every verb, plain / JSON / XML bodies
http('/books')                          // GET
httpPostJson('/books', [title: 'Geb'])
httpPutXml('/books/1') { book { title('Geb') } }
httpPatchJson(…) · httpDelete(…) · httpHead(…)

// form fields, new in 8.0
httpPostForm('/hello', [name: 'Ada'])

// multipart
httpPostMultipart('/upload', MultipartBody.builder()
        .addPart('name', 'cover')
        .addPart('file', 'cover.png', 'image/png', bytes)
        .build())

// a custom client, built from the defaults
def client = newHttpClientWith { followRedirects(NEVER) }
http('/old-page', client).assertStatus(302)

// keep cookies, e.g. a login session
def withCookies = newHttpClientWith {
    cookieHandler(new CookieManager())
}
```

```
response
    .assertStatus(200)
    .assertHeadersIgnoreCase('content-type': 'application/json')
    .assertJson([id: 1, title: 'Geb'])          // whole tree
    .assertJsonContains([title: 'Geb'])         // subset
    .assertContains('Geb')
    .assertNotContains('error')

response.json()      // Map
response.jsonList()  // List
response.xml()       // GPathResult
```

`HttpClientSupport`: more of the API

```
// every verb, plain / JSON / XML bodies
http('/books')                          // GET
httpPostJson('/books', [title: 'Geb'])
httpPutXml('/books/1') { book { title('Geb') } }
httpPatchJson(…) · httpDelete(…) · httpHead(…)

// form fields, new in 8.0
httpPostForm('/hello', [name: 'Ada'])

// multipart
httpPostMultipart('/upload', MultipartBody.builder()
        .addPart('name', 'cover')
        .addPart('file', 'cover.png', 'image/png', bytes)
        .build())

// a custom client, built from the defaults
def client = newHttpClientWith { followRedirects(NEVER) }
http('/old-page', client).assertStatus(302)

// keep cookies, e.g. a login session
def withCookies = newHttpClientWith {
    cookieHandler(new CookieManager())
}
```

```
response
    .assertStatus(200)
    .assertHeadersIgnoreCase('content-type': 'application/json')
    .assertJson([id: 1, title: 'Geb'])          // whole tree
    .assertJsonContains([title: 'Geb'])         // subset
    .assertContains('Geb')
    .assertNotContains('error')

response.json()      // Map
response.jsonList()  // List
response.xml()       // GPathResult
```

- Relative paths go to the running app: `localhost:<serverPort>` plus context path

`HttpClientSupport`: more of the API

```
// every verb, plain / JSON / XML bodies
http('/books')                          // GET
httpPostJson('/books', [title: 'Geb'])
httpPutXml('/books/1') { book { title('Geb') } }
httpPatchJson(…) · httpDelete(…) · httpHead(…)

// form fields, new in 8.0
httpPostForm('/hello', [name: 'Ada'])

// multipart
httpPostMultipart('/upload', MultipartBody.builder()
        .addPart('name', 'cover')
        .addPart('file', 'cover.png', 'image/png', bytes)
        .build())

// a custom client, built from the defaults
def client = newHttpClientWith { followRedirects(NEVER) }
http('/old-page', client).assertStatus(302)

// keep cookies, e.g. a login session
def withCookies = newHttpClientWith {
    cookieHandler(new CookieManager())
}
```

```
response
    .assertStatus(200)
    .assertHeadersIgnoreCase('content-type': 'application/json')
    .assertJson([id: 1, title: 'Geb'])          // whole tree
    .assertJsonContains([title: 'Geb'])         // subset
    .assertContains('Geb')
    .assertNotContains('error')

response.json()      // Map
response.jsonList()  // List
response.xml()       // GPathResult
```

- Relative paths go to the running app: `localhost:<serverPort>` plus context path
- One shared JDK client: redirects followed, 60 s timeouts, `grails.http.client.timeout` to change

MongoDB and Redis in a container 7.0

```
testImplementation 'org.apache.grails.testing:grails-testing-support-mongodb'
```

```
// ./gradlew test, no app: your spec gets host and port
class BookSpec extends AutoStartedMongoSpec {

    @Shared
    @AutoCleanup
    MongoDatastore datastore

    void setupSpec() {
        datastore = new MongoDatastore(Book,
                'grails.mongodb.host': mongoHost,
                'grails.mongodb.port': mongoPort)
    }

    void 'GORM writes a real MongoDB document'() {
        when: 'a book is saved through GORM'
            new Book(title: 'Geb').save(flush: true)

        then: 'the MongoDB driver finds it'
            Book.collection.find().first().title == 'Geb'
    }
}
```

MongoDB and Redis in a container 7.0

```
testImplementation 'org.apache.grails.testing:grails-testing-support-mongodb'
```

```
// ./gradlew test, no app: your spec gets host and port
class BookSpec extends AutoStartedMongoSpec {

    @Shared
    @AutoCleanup
    MongoDatastore datastore

    void setupSpec() {
        datastore = new MongoDatastore(Book,
                'grails.mongodb.host': mongoHost,
                'grails.mongodb.port': mongoPort)
    }

    void 'GORM writes a real MongoDB document'() {
        when: 'a book is saved through GORM'
            new Book(title: 'Geb').save(flush: true)

        then: 'the MongoDB driver finds it'
            Book.collection.find().first().title == 'Geb'
    }
}
```

- **In** `integrationTest` **:** an ordinary `@Integration` spec. The module on the classpath is enough: one container per run, handed to the app through `grails.mongodb.host` / `port` . Already running on `27017` ? It uses that instead

MongoDB and Redis in a container 7.0

```
testImplementation 'org.apache.grails.testing:grails-testing-support-mongodb'
```

```
// ./gradlew test, no app: your spec gets host and port
class BookSpec extends AutoStartedMongoSpec {

    @Shared
    @AutoCleanup
    MongoDatastore datastore

    void setupSpec() {
        datastore = new MongoDatastore(Book,
                'grails.mongodb.host': mongoHost,
                'grails.mongodb.port': mongoPort)
    }

    void 'GORM writes a real MongoDB document'() {
        when: 'a book is saved through GORM'
            new Book(title: 'Geb').save(flush: true)

        then: 'the MongoDB driver finds it'
            Book.collection.find().first().title == 'Geb'
    }
}
```

- **In** `integrationTest` : an ordinary `@Integration` spec. The module on the classpath is enough: one container per run, handed to the app through `grails.mongodb.host` / `port` . Already running on `27017` ? It uses that instead
- Pin the image with the `mongodbContainerVersion` system property

MongoDB and Redis in a container 7.0

```
testImplementation 'org.apache.grails.testing:grails-testing-support-mongodb'
```

```
// ./gradlew test, no app: your spec gets host and port
class BookSpec extends AutoStartedMongoSpec {

    @Shared
    @AutoCleanup
    MongoDatastore datastore

    void setupSpec() {
        datastore = new MongoDatastore(Book,
                'grails.mongodb.host': mongoHost,
                'grails.mongodb.port': mongoPort)
    }

    void 'GORM writes a real MongoDB document'() {
        when: 'a book is saved through GORM'
            new Book(title: 'Geb').save(flush: true)

        then: 'the MongoDB driver finds it'
            Book.collection.find().first().title == 'Geb'
    }
}
```

- **In** `integrationTest` : an ordinary `@Integration` spec. The module on the classpath is enough: one container per run, handed to the app through `grails.mongodb.host` / `port` . Already running on `27017` ? It uses that instead
- Pin the image with the `mongodbContainerVersion` system property
- `EmbeddedReplicaSetSpec` 8.0: a real `mongod` **without Docker**, as a one-node replica set, so transactions work

MongoDB and Redis in a container 7.0

```
testImplementation 'org.apache.grails.testing:grails-testing-support-mongodb'
```

```
// ./gradlew test, no app: your spec gets host and port
class BookSpec extends AutoStartedMongoSpec {

    @Shared
    @AutoCleanup
    MongoDatastore datastore

    void setupSpec() {
        datastore = new MongoDatastore(Book,
                'grails.mongodb.host': mongoHost,
                'grails.mongodb.port': mongoPort)
    }

    void 'GORM writes a real MongoDB document'() {
        when: 'a book is saved through GORM'
            new Book(title: 'Geb').save(flush: true)

        then: 'the MongoDB driver finds it'
            Book.collection.find().first().title == 'Geb'
    }
}
```

- **In** `integrationTest` : an ordinary `@Integration` spec. The module on the classpath is enough: one container per run, handed to the app through `grails.mongodb.host` / `port` . Already running on `27017` ? It uses that instead
- Pin the image with the `mongodbContainerVersion` system property
- `EmbeddedReplicaSetSpec` 8.0: a real `mongod` **without Docker**, as a one-node replica set, so transactions work
- **Redis** 8.0.1: `grails-testing-support-redis` , same group and idea, sets `REDIS_HOST` / `REDIS_PORT` ; expected on Maven Central with 8.0.1

Browser tests with Geb

Still `@Integration`, just from the outside with a real browser

What changed in Grails 7

Before: a browser on every machine

```
// build.gradle, generated by Grails 6.0.0
plugins {
    id 'com.github.erdi.webdriver-binaries' version '3.0'
}
webdriverBinaries {
    chromedriver '110.0.5481.77'
    geckodriver '0.32.2'
}
```

- Chrome updates itself, the pinned driver doesn’t
- Every laptop and CI image needs the right browser installed
- A `GebConfig.groovy` driver per browser, picked with `-Dgeb.env`

Now: the browser in a container

What changed in Grails 7

Before: a browser on every machine

```
// build.gradle, generated by Grails 6.0.0
plugins {
    id 'com.github.erdi.webdriver-binaries' version '3.0'
}
webdriverBinaries {
    chromedriver '110.0.5481.77'
    geckodriver '0.32.2'
}
```

- Chrome updates itself, the pinned driver doesn't
- Every laptop and CI image needs the right browser installed
- A `GebConfig.groovy` driver per browser, picked with `-Dgeb.env`

Now: the browser in a container

- `ContainerGebSpec` : same Geb API, browser in a **Testcontainers** Selenium container

What changed in Grails 7

Before: a browser on every machine

```
// build.gradle, generated by Grails 6.0.0
plugins {
    id 'com.github.erdi.webdriver-binaries' version '3.0'
}
webdriverBinaries {
    chromedriver '110.0.5481.77'
    geckodriver '0.32.2'
}
```

- Chrome updates itself, the pinned driver doesn't
- Every laptop and CI image needs the right browser installed
- A `GebConfig.groovy` driver per browser, picked with `-Dgeb.env`

Now: the browser in a container

- `ContainerGebSpec` : same Geb API, browser in a **Testcontainers** Selenium container
- The image follows your Selenium version: browser and driver always match

What changed in Grails 7

Before: a browser on every machine

```
// build.gradle, generated by Grails 6.0.0
plugins {
    id 'com.github.erdi.webdriver-binaries' version '3.0'
}
webdriverBinaries {
    chromedriver '110.0.5481.77'
    geckodriver '0.32.2'
}
```

- Chrome updates itself, the pinned driver doesn't
- Every laptop and CI image needs the right browser installed
- A `GebConfig.groovy` driver per browser, picked with `-Dgeb.env`

Now: the browser in a container

- `ContainerGebSpec` : same Geb API, browser in a **Testcontainers** Selenium container
- The image follows your Selenium version: browser and driver always match
- Only Docker needed, the same browser everywhere

What changed in Grails 7

Before: a browser on every machine

```
// build.gradle, generated by Grails 6.0.0
plugins {
    id 'com.github.erdi.webdriver-binaries' version '3.0'
}
webdriverBinaries {
    chromedriver '110.0.5481.77'
    geckodriver '0.32.2'
}
```

- Chrome updates itself, the pinned driver doesn't
- Every laptop and CI image needs the right browser installed
- A `GebConfig.groovy` driver per browser, picked with `-Dgeb.env`

Now: the browser in a container

- `ContainerGebSpec` : same Geb API, browser in a **Testcontainers** Selenium container
- The image follows your Selenium version: browser and driver always match
- Only Docker needed, the same browser everywhere
- Since Dec 2024: Grails 6 (Geb plugin 4.1) and 7, the Forge default

What changed in Grails 7

Before: a browser on every machine

```
// build.gradle, generated by Grails 6.0.0
plugins {
    id 'com.github.erdi.webdriver-binaries' version '3.0'
}
webdriverBinaries {
    chromedriver '110.0.5481.77'
    geckodriver '0.32.2'
}
```

- Chrome updates itself, the pinned driver doesn't
- Every laptop and CI image needs the right browser installed
- A `GebConfig.groovy` driver per browser, picked with `-Dgeb.env`

Now: the browser in a container

- `ContainerGebSpec` : same Geb API, browser in a **Testcontainers** Selenium container
- The image follows your Selenium version: browser and driver always match
- Only Docker needed, the same browser everywhere
- Since Dec 2024: Grails 6 (Geb plugin 4.1) and 7, the Forge default
- `GebSpec` still works, with the browsers on your machine

What changed in Grails 7

Before: a browser on every machine

```
// build.gradle, generated by Grails 6.0.0
plugins {
    id 'com.github.erdi.webdriver-binaries' version '3.0'
}
webdriverBinaries {
    chromedriver '110.0.5481.77'
    geckodriver '0.32.2'
}
```

- Chrome updates itself, the pinned driver doesn't
- Every laptop and CI image needs the right browser installed
- A `GebConfig.groovy` driver per browser, picked with `-Dgeb.env`

Now: the browser in a container

- `ContainerGebSpec` : same Geb API, browser in a **Testcontainers** Selenium container
- The image follows your Selenium version: browser and driver always match
- Only Docker needed, the same browser everywhere
- Since Dec 2024: Grails 6 (Geb plugin 4.1) and 7, the Forge default
- `GebSpec` still works, with the browsers on your machine
- Now in Apache Geb as `geb-testcontainers` , not released yet

GebSpec → ContainerGebSpec

```
import geb.spock.GebSpec
import grails.testing.mixin.integration.Integration

@Integration
class HomeSpec extends GebSpec {

    void 'home page renders'() {
        when:
            go('/')

        then:
            waitFor { title == 'Welcome to Grails' }
    }
}
```

GebSpec → ContainerGebSpec

```groovy
import grails.plugin.geb.ContainerGebSpec
import grails.testing.mixin.integration.Integration

@Integration
class HomeSpec extends ContainerGebSpec {

    void 'home page renders'() {
        when:
            go('/')

        then:
            waitFor { title == 'Welcome to Grails' }
    }
}
```

What `ContainerGebSpec` brings

Recording & reporting

```
tasks.named('integrationTest') {
    systemProperty('grails.geb.recording.mode', 'RECORD_FAILING')
}
```

`SKIP` · `RECORD_ALL` · `RECORD_FAILING` → `build/gebContainer/recordings/*.mp4`

`@ContainerGebConfiguration(reporting = true)` → screenshots & page source in `build/gebContainer/reports`

Host & context path

```
@ContainerGebConfiguration(hostName = 'testing.example.com')
```

`server.servlet.context-path` is picked up automatically

Timeouts & waiting

System properties instead of `GebConfig.groovy` edits: `grails.geb.timeouts.timeout`, `retryInterval`, `pageLoad`, `implicitlyWait`, `grails.geb.atCheckWaiting.enabled`

Uploads

`uploadFile('/tmp/data.csv')` copies the file into the browser container first

Your own driver

A `GebConfig.groovy` on the test classpath still wins: Firefox, another Selenium image, your own `RemoteWebDriver`

A real browser spec

```groovy
@Integration
class HomeSpec extends ContainerGebSpec {

    void 'the home page renders in a containerized browser'() {
        when: 'visiting the home page'
            def page = to(HomePage)

        then: 'the heading is there'
            page.heading.contains('Welcome')
    }

    void '/people lists people created directly through GORM'() {
        given: 'the app runs in the same JVM, so GORM is right here'
            Person.withNewTransaction {
                new Person(firstName: 'Ada', lastName: 'Lovelace')
                        .save(flush: true)
            }

        when: 'visiting the people page'
            def page = to(PeoplePage)

        then: 'the person created through GORM is listed'
            page.people == ['Ada Lovelace']
    }
}
```

```groovy
class HomePage extends Page {

    static url = '/'
    static at = {
        title == 'Welcome to Grails'
    }

    static content = {
        heading { $('h1').text() }
    }
}
```

A real browser spec

```groovy
@Integration
class HomeSpec extends ContainerGebSpec {

    void 'the home page renders in a containerized browser'() {
        when: 'visiting the home page'
            def page = to(HomePage)

        then: 'the heading is there'
            page.heading.contains('Welcome')
    }

    void '/people lists people created directly through GORM'() {
        given: 'the app runs in the same JVM, so GORM is right here'
            Person.withNewTransaction {
                new Person(firstName: 'Ada', lastName: 'Lovelace')
                        .save(flush: true)
            }

        when: 'visiting the people page'
            def page = to(PeoplePage)

        then: 'the person created through GORM is listed'
            page.people == ['Ada Lovelace']
    }
}
```

```groovy
class HomePage extends Page {

    static url = '/'
    static at = {
        title == 'Welcome to Grails'
    }

    static content = {
        heading { $('h1').text() }
    }
}
```

A real browser spec

```groovy
@Integration
class HomeSpec extends ContainerGebSpec {

    void 'the home page renders in a containerized browser'() {
        when: 'visiting the home page'
            def page = to(HomePage)

        then: 'the heading is there'
            page.heading.contains('Welcome')
    }

    void '/people lists people created directly through GORM'() {
        given: 'the app runs in the same JVM, so GORM is right here'
            Person.withNewTransaction {
                new Person(firstName: 'Ada', lastName: 'Lovelace')
                        .save(flush: true)
            }

        when: 'visiting the people page'
            def page = to(PeoplePage)

        then: 'the person created through GORM is listed'
            page.people == ['Ada Lovelace']
    }
}
```

```groovy
class HomePage extends Page {

    static url = '/'
    static at = {
        title == 'Welcome to Grails'
    }

    static content = {
        heading { $('h1').text() }
    }
}
```

A real browser spec

```groovy
@Integration
class HomeSpec extends ContainerGebSpec {

    void 'the home page renders in a containerized browser'() {
        when: 'visiting the home page'
            def page = to(HomePage)

        then: 'the heading is there'
            page.heading.contains('Welcome')
    }

    void '/people lists people created directly through GORM'() {
        given: 'the app runs in the same JVM, so GORM is right here'
            Person.withNewTransaction {
                new Person(firstName: 'Ada', lastName: 'Lovelace')
                        .save(flush: true)
            }

        when: 'visiting the people page'
            def page = to(PeoplePage)

        then: 'the person created through GORM is listed'
            page.people == ['Ada Lovelace']
    }
}
```

```groovy
class HomePage extends Page {

    static url = '/'
    static at = {
        title == 'Welcome to Grails'
    }

    static content = {
        heading { $('h1').text() }
    }
}
```

Same JVM as the app: `Person.withNewTransaction { … }` sets up data through GORM, no fixtures API needed.

A real browser spec

```groovy
@Integration
class HomeSpec extends ContainerGebSpec {

    void 'the home page renders in a containerized browser'() {
        when: 'visiting the home page'
            def page = to(HomePage)

        then: 'the heading is there'
            page.heading.contains('Welcome')
    }

    void '/people lists people created directly through GORM'() {
        given: 'the app runs in the same JVM, so GORM is right here'
            Person.withNewTransaction {
                new Person(firstName: 'Ada', lastName: 'Lovelace')
                        .save(flush: true)
            }

        when: 'visiting the people page'
            def page = to(PeoplePage)

        then: 'the person created through GORM is listed'
            page.people == ['Ada Lovelace']
    }
}
```

```groovy
class HomePage extends Page {

    static url = '/'
    static at = {
        title == 'Welcome to Grails'
    }

    static content = {
        heading { $('h1').text() }
    }
}
```

Same JVM as the app: `Person.withNewTransaction { … }` sets up data through GORM, no fixtures API needed.

A real browser spec

```groovy
@Integration
class HomeSpec extends ContainerGebSpec {

    void 'the home page renders in a containerized browser'() {
        when: 'visiting the home page'
            def page = to(HomePage)

        then: 'the heading is there'
            page.heading.contains('Welcome')
    }

    void '/people lists people created directly through GORM'() {
        given: 'the app runs in the same JVM, so GORM is right here'
            Person.withNewTransaction {
                new Person(firstName: 'Ada', lastName: 'Lovelace')
                        .save(flush: true)
            }

        when: 'visiting the people page'
            def page = to(PeoplePage)

        then: 'the person created through GORM is listed'
            page.people == ['Ada Lovelace']
    }
}
```

```groovy
class HomePage extends Page {

    static url = '/'
    static at = {
        title == 'Welcome to Grails'
    }

    static content = {
        heading { $('h1').text() }
    }
}
```

Same JVM as the app: `Person.withNewTransaction { … }` sets up data through GORM, no fixtures API needed.

Simulated latency

NEW IN 8.0

```
// build.gradle
dependencies {
    functionalTestRuntimeOnly '…:grails-testing-support-latency'
}
tasks.named('functionalTest') {
    if (project.hasProperty('latency')) {
        systemProperty('grails.testing.latency.enabled', 'true')
    }
}
```

```
# application.yml, test environment
grails.testing.latency:
    min-delay: 250ms
    max-delay: 2s          # the default
    probability: 0.5       # delay half the requests
    url-patterns: /api/*   # default /*
    seed: 42               # reproducible sequence
```

A servlet filter, first in the chain, sleeps a random time before each matching request.

```
./gradlew functionalTest -Platency

Delaying GET / by 1559ms
Delaying GET /assets/…/bootstrap.css by 1495ms
Delaying GET /assets/grails.svg by 1866ms
…  34 requests delayed, both specs still green
```

Simulated latency NEW IN 8.0

```
// build.gradle
dependencies {
    functionalTestRuntimeOnly '…:grails-testing-support-latency'
}
tasks.named('functionalTest') {
    if (project.hasProperty('latency')) {
        systemProperty('grails.testing.latency.enabled', 'true')
    }
}
```

```
# application.yml, test environment
grails.testing.latency:
    min-delay: 250ms
    max-delay: 2s          # the default
    probability: 0.5       # delay half the requests
    url-patterns: /api/*   # default /*
    seed: 42               # reproducible sequence
```

A servlet filter, first in the chain, sleeps a random time before each matching request.

```
./gradlew functionalTest -Platency

Delaying GET / by 1559ms
Delaying GET /assets/…/bootstrap.css by 1495ms
Delaying GET /assets/grails.svg by 1866ms
…  34 requests delayed, both specs still green
```

- A test that races a click-triggered navigation now **fails every time**, not now and then on CI

Simulated latency NEW IN 8.0

```
// build.gradle
dependencies {
    functionalTestRuntimeOnly '…:grails-testing-support-latency'
}
tasks.named('functionalTest') {
    if (project.hasProperty('latency')) {
        systemProperty('grails.testing.latency.enabled', 'true')
    }
}
```

```
# application.yml, test environment
grails.testing.latency:
    min-delay: 250ms
    max-delay: 2s          # the default
    probability: 0.5       # delay half the requests
    url-patterns: /api/*   # default /*
    seed: 42               # reproducible sequence
```

A servlet filter, first in the chain, sleeps a random time before each matching request.

```
./gradlew functionalTest -Platency

Delaying GET / by 1559ms
Delaying GET /assets/…/bootstrap.css by 1495ms
Delaying GET /assets/grails.svg by 1866ms
…  34 requests delayed, both specs still green
```

- A test that races a click-triggered navigation now **fails every time**, not now and then on CI
- **Never enable it in production config**

Slicing the test suite

Phases split tests by kind, tags split them within a phase

`testPhases { }`: one line, a whole phase 7.1

```
// build.gradle
testPhases {
    functionalTest { }
}

dependencies {
    functionalTestImplementation testFixtures('…:grails-geb')
}
```

Grails ships `test` and `integrationTest`, nothing else. Since 7.1 `integrationTest` is itself built on `testPhases`, so everything here applies to it too.

That one line gives you:

`testPhases { }`: one line, a whole phase 7.1

```
// build.gradle
testPhases {
    functionalTest { }
}

dependencies {
    functionalTestImplementation testFixtures('…:grails-geb')
}
```

Grails ships `test` and `integrationTest`, nothing else. Since 7.1 `integrationTest` is itself built on `testPhases`, so everything here applies to it too.

That one line gives you:

- a source set in `src/functional-test/groovy`

`testPhases { }`: one line, a whole phase 7.1

```
// build.gradle
testPhases {
    functionalTest { }
}

dependencies {
    functionalTestImplementation testFixtures('…:grails-geb')
}
```

Grails ships `test` and `integrationTest`, nothing else. Since 7.1 `integrationTest` is itself built on `testPhases`, so everything here applies to it too.

That one line gives you:

- a source set in `src/functional-test/groovy`
- `functionalTestImplementation` and `functionalTestRuntimeOnly`, extending the test configurations

`testPhases { }`: one line, a whole phase 7.1

```
// build.gradle
testPhases {
    functionalTest { }
}

dependencies {
    functionalTestImplementation testFixtures('…:grails-geb')
}
```

Grails ships `test` and `integrationTest`, nothing else. Since 7.1 `integrationTest` is itself built on `testPhases`, so everything here applies to it too.

That one line gives you:

- a source set in `src/functional-test/groovy`
- `functionalTestImplementation` and `functionalTestRuntimeOnly`, extending the test configurations
- a `functionalTest` task, wired into `check`

`testPhases { }`: one line, a whole phase 7.1

```
// build.gradle
testPhases {
    functionalTest { }
}

dependencies {
    functionalTestImplementation testFixtures('…:grails-geb')
}
```

Grails ships `test` and `integrationTest`, nothing else. Since 7.1 `integrationTest` is itself built on `testPhases`, so everything here applies to it too.

That one line gives you:

- a source set in `src/functional-test/groovy`
- `functionalTestImplementation` and `functionalTestRuntimeOnly`, extending the test configurations
- a `functionalTest` task, wired into `check`
- a system property, `is.grails.functional.test=true`

`testPhases { }`: one line, a whole phase 7.1

```
// build.gradle
testPhases {
    functionalTest { }
}

dependencies {
    functionalTestImplementation testFixtures('…:grails-geb')
}
```

Grails ships `test` and `integrationTest`, nothing else. Since 7.1 `integrationTest` is itself built on `testPhases`, so everything here applies to it too.

That one line gives you:

- a source set in `src/functional-test/groovy`
- `functionalTestImplementation` and `functionalTestRuntimeOnly`, extending the test configurations
- a `functionalTest` task, wired into `check`
- a system property, `is.grails.functional.test=true`
- IntelliJ marking the folder as test sources

`testPhases { }`: one line, a whole phase 7.1

```
// build.gradle
testPhases {
    functionalTest { }
}

dependencies {
    functionalTestImplementation testFixtures('…:grails-geb')
}
```

Grails ships `test` and `integrationTest`, nothing else. Since 7.1 `integrationTest` is itself built on `testPhases`, so everything here applies to it too.

That one line gives you:

- a source set in `src/functional-test/groovy`
- `functionalTestImplementation` and `functionalTestRuntimeOnly`, extending the test configurations
- a `functionalTest` task, wired into `check`
- a system property, `is.grails.functional.test=true`
- IntelliJ marking the folder as test sources
- **your `src/test` classes on the classpath**, so shared base specs and helpers just work

Customize, detect and run a phase

```
// build.gradle
testPhases {
    smokeTest {
        sourceFolderName = 'src/smoke-tests'
        systemPropertyName = 'is.smoke.test'
        ideaIntegration = false
    }
}
```

Property	Default
`sourceFolderName`	`src/smoke-test`
`systemPropertyName`	`is.grails.smoke.test`
`ideaIntegration`	`true`

```
// shared test or app code
if (Boolean.getBoolean('is.grails.functional.test')) {
    // e.g. another datasource, shorter timeouts,
    // an interceptor switched off
}
```

```
./gradlew smokeTest
./gradlew functionalTest --tests 'demo.HomeSpec'
./gradlew check            # every phase
```

One merged HTML report for all phases:
`build/reports/tests/index.html`

Test slicing with JUnit Platform tags

```
import spock.lang.Tag   // Spock's mapping of JUnit Platform tags

// ContainerGebSpec already carries @Tag('geb') and @Tag('container-geb')
@Tag('slow')
@Integration
class BigReportSpec extends ContainerGebSpec { … }
```

```
// build.gradle: pick what runs
tasks.named('functionalTest') {
    useJUnitPlatform {
        if (project.hasProperty('excludeTestTags')) {
            excludeTags(project.excludeTestTags.split(','))
        }
    }
}
```

```
./gradlew functionalTest -PexcludeTestTags=slow
```

Upgrading and what’s next

What to change, what to watch

Upgrade cheat-sheet

6 → 7

- Java 17, Groovy 4, `jakarta.*`
- Test JVM args are no longer set for you: configure them on the `Test` task
- Test deps are no longer on the production classpath
- Optional: `GebSpec` → `ContainerGebSpec`, needs a container runtime
- Testcontainers 1.x coordinates still fine in 7.0/7.1
- Adopt: `@DatabaseCleanup` (7.0.8), `HttpClientSupport` (7.1), `testPhases { }` (7.1)

7 → 8

- Java 21, Groovy 5, Spock **2.4-groovy-5.0**
- GORM properties **nullable by default**: revisit validation specs, or set `grails.gorm.default.nullable: false`
- `@MockBean` → `@MockitoBean`, `@SpyBean` → `@MockitoSpyBean`
- `doWithSpring()` is deprecated: declare a `beans` block instead
- Testcontainers 2.x: `testcontainers-selenium` coordinates
- Delete `getPurgeTagLibMetaClass()` overrides, cleanup is automatic

> Most apps: bump Java, recheck validation specs. The traits and `GebSpec` keep working.

Test helpers live in test fixtures

```
// build.gradle: only the fixtures you use, only on test configurations
dependencies {
    testImplementation testFixtures('org.apache.grails:grails-core')
    testImplementation testFixtures('org.apache.grails.web:grails-web-common')
    testImplementation testFixtures('org.apache.grails:grails-views-gson')
    integrationTestImplementation testFixtures('org.apache.grails:grails-geb')
}
```

Test helpers live in test fixtures

```
// build.gradle: only the fixtures you use, only on test configurations
dependencies {
    testImplementation testFixtures('org.apache.grails:grails-core')
    testImplementation testFixtures('org.apache.grails.web:grails-web-common')
    testImplementation testFixtures('org.apache.grails:grails-views-gson')
    integrationTestImplementation testFixtures('org.apache.grails:grails-geb')
}
```

- Test-only classes are moving out of each module's main jar into its **Gradle test fixtures**, so they stay off your app's runtime classpath

Test helpers live in test fixtures

```
// build.gradle: only the fixtures you use, only on test configurations
dependencies {
    testImplementation testFixtures('org.apache.grails:grails-core')
    testImplementation testFixtures('org.apache.grails.web:grails-web-common')
    testImplementation testFixtures('org.apache.grails:grails-views-gson')
    integrationTestImplementation testFixtures('org.apache.grails:grails-geb')
}
```

- Test-only classes are moving out of each module's main jar into its **Gradle test fixtures**, so they stay off your app's runtime classpath
- Grails 7: `grails-geb` ships `ContainerGebSpec` this way

Test helpers live in test fixtures

```
// build.gradle: only the fixtures you use, only on test configurations
dependencies {
    testImplementation testFixtures('org.apache.grails:grails-core')
    testImplementation testFixtures('org.apache.grails.web:grails-web-common')
    testImplementation testFixtures('org.apache.grails:grails-views-gson')
    integrationTestImplementation testFixtures('org.apache.grails:grails-geb')
}
```

- Test-only classes are moving out of each module's main jar into its **Gradle test fixtures**, so they stay off your app's runtime classpath
- Grails 7: `grails-geb` ships `ContainerGebSpec` this way
- Grails 8: `MockGrailsPluginManager`, `MockResourceLoader` and friends, `GrailsWebMockUtil`, `JsonViewTest`, the GraphQL `GraphQLSchemaSpec`

Test helpers live in test fixtures

```
// build.gradle: only the fixtures you use, only on test configurations
dependencies {
    testImplementation testFixtures('org.apache.grails:grails-core')
    testImplementation testFixtures('org.apache.grails.web:grails-web-common')
    testImplementation testFixtures('org.apache.grails:grails-views-gson')
    integrationTestImplementation testFixtures('org.apache.grails:grails-geb')
}
```

- Test-only classes are moving out of each module's main jar into its **Gradle test fixtures**, so they stay off your app's runtime classpath
- Grails 7: `grails-geb` ships `ContainerGebSpec` this way
- Grails 8: `MockGrailsPluginManager`, `MockResourceLoader` and friends, `GrailsWebMockUtil`, `JsonViewTest`, the GraphQL `GraphQLSchemaSpec`
- Your tests still compile against them with `testFixtures(…)`: versions come from the Grails BOM. Never on `implementation`

What's next

Spock 2.4 IN GRAILS 8

- `@Verify` / `@VerifyAll` helper methods, Mockito mock maker: final classes, `SpyStatic()`
- `filter:` blocks, Cartesian products, `@Snapshot`, `verifyEach()`

Spock 2.5 6 OCT 2026

- Groovy 6 and Spring 7 support
- `final` local variables in `where:` blocks
- `TooManyInvocationsError` explains argument mismatches
- Works in Grails 8 today: `2.5-groovy-5.0`

Apache Geb

- `geb-playwright` MERGED, NOT RELEASED: an experimental Playwright-backed `WebDriver`, so your existing pages, modules and `waitFor` run on Playwright instead of Selenium

Thank you

Slides & demo app

github.com/matrei/grails-testing-revisited

Questions?